COOPERATIVA HUERTA DE MURCIA · BASES DE DATOS

Misión SQL

El enigma de la Cooperativa Huerta de Murcia

Programación y Bases de Datos · Bloque 3: Bases de datos

Módulo 0484 *Bases de datos* · 1.º de los CFGS de DAM y DAW



La tormenta

Una tormenta de granizo ha dañado el servidor de la **Cooperativa Huerta de Murcia**, justo antes de la campaña de exportación.

La gerencia te ha contratado como **técnico o técnica junior de bases de datos** para recuperar la información de **socios, productos y entregas**.

> Pero el acceso a la copia de seguridad está protegido por una **caja fuerte digital con un código de cuatro dígitos**.



Tu misión

1 · Supera los juegos

Son tres juegos. Cada uno que superes revela uno o dos dígitos: la **llave** del juego.

2 · Reúne los dígitos

Anota cada dígito en su casilla. Necesitas los **cuatro** para abrir la caja fuerte.

3 · Abre la caja fuerte

En la última diapositiva introduce el código y **salva la campaña**.



Equipos y normas del juego

- Formad **equipos**; cada uno tiene su perfil.
- Marcador, insignias y dígitos **se guardan en tu navegador**.
- Los juegos se desbloquean **en orden**.
- Puedes **repetirlos** cuando quieras.

Sin registro

- No hay cuentas.
- No se recogen datos personales.
- El juego 3 ejecuta **SQL real** en tu navegador: no se instala nada.



Puntos e insignias

Juego	Cómo se puntúa
1 · Clasifica y ordena	**+10** por acierto, **–2** por fallo (con una explicación breve)
2 · ¿Qué devuelve la consulta?	**500–1000** por acierto, según la rapidez (30 s por pregunta). **+100** de bonificación por racha a partir del 3.er acierto seguido
3 · Terminal de la Cooperativa	**500** por incidencia. Cada pista (máximo 2) resta **100** y cada comprobación fallida, **25**. Mínimo **150**

Insignias que puedes ganar: **Clasificador** **Detective de consultas** **Maestro del DML**

El itinerario de la misión

Presentación **Misión, base de datos y SQL** › Juego 1 **Clasifica y ordena** dígito 1 › Presentación **Consultas, resumen, varias tablas y subconsultas** › Juego 2 **¿Qué devuelve la consulta?** dígito 2 › Presentación **Manipulación de datos y transacciones** › Juego 3 **Terminal de la Cooperativa** dígitos 3 y 4

La presentación es el **tablero**: desde aquí se accede a cada juego y aquí se vuelve con el dígito de la llave. Al final, la **caja fuerte**.

Las tres sesiones (55 minutos cada una)

Sesión 1

- Misión y base de datos
- Introducción a SQL
- **Juego 1**
- Consultas simples y de resumen

Sesión 2

- Varias tablas: composiciones
- Subconsultas
- **Juego 2**, por equipos y con clasificación proyectada

Sesión 3

- Manipulación de datos
- Transacciones
- **Juego 3**, por parejas
- Apertura de la **caja fuerte**
- Puesta en común y autoevaluación

¿Qué vamos a aprender?

1. Distinguir los **sublenguajes** de SQL (DDL, DML, DCL y TCL) y clasificar sus sentencias.
2. Hacer **consultas simples** con `WHERE`, `ORDER BY` y `DISTINCT`, usando operadores, `BETWEEN`, `IN`, `LIKE` e `IS NULL`.
3. Obtener **resúmenes** con funciones de agregado, `GROUP BY` y `HAVING`.
4. Combinar **varias tablas** con `INNER JOIN` y `LEFT` / `RIGHT JOIN`.
5. Resolver consultas con **subconsultas** y compararlas con una solución equivalente con `JOIN`.
6. **Insertar, actualizar y borrar** datos respetando la integridad referencial.
7. Entender las **transacciones**: `COMMIT`, `ROLLBACK` y `SAVEPOINT`.
8. Tener **autonomía y constancia** ante los errores, y colaborar con tu equipo.

Primera parte

La base de datos

Conoce la base de datos **cooperativa** y el lenguaje **SQL**.



La base de datos `cooperativa`

Todos los retos se resuelven sobre la misma base de datos, con **tres tablas**:

socio

- id_socio
- nombre
- municipio
- fecha_alta

producto

- id_producto
- nombre
- categoria
- precio_kg

entrega

- id_entrega
- id_socio FK
- id_producto FK
- fecha
- kilos

Un **socio** hace muchas **entregas** y un **producto** aparece en muchas **entregas**. Las claves primarias van subrayadas; FK = clave ajena.



Tabla `socio` y tabla `producto`

id	nombre	municipio	fecha_alta
1	Ana Martínez	Murcia	2019-02-11
2	José López	Lorca	2020-05-03
3	Carmen Sánchez	Cartagena	2021-09-20
4	Pedro Ruiz	Murcia	2022-01-15
5	Lucía Gómez	Cieza	2025-11-30

id	nombre	categoria	precio_kg
1	Limón	Cítrico	0,60
2	Naranja	Cítrico	0,45
3	Pimiento	Hortaliza	1,20
4	Alcachofa	Hortaliza	1,80
5	Melocotón	Fruta de hueso	1,50
6	Albaricoque	Fruta de hueso	2,10

Tabla `entrega`

id_entrega	id_socio	id_producto	fecha	kilos
1	1	1	2026-03-02	500
2	1	3	2026-03-05	200
3	2	2	2026-03-05	800
4	2	4	2026-03-10	150
5	3	1	2026-03-12	300
6	3	5	2026-06-20	400
7	4	3	2026-03-15	250
8	4	4	2026-03-18	100
9	1	5	2026-06-25	350

¿Qué es SQL?

SQL (*Structured Query Language*) es el lenguaje estándar para **definir, consultar y manipular** la información de una base de datos relacional.

- Basado en el **modelo relacional** de Codd (1970).
- Nació en IBM como **SEQUEL** (1974).
- Normalizado en **ISO/IEC 9075**.
- **Declarativo**: dices *qué* quieres, no *cómo*.

SGBD habituales

- **MySQL / MariaDB**
- **PostgreSQL**
- **Oracle**
- **SQLite**: el que usa el juego 3, dentro de tu navegador

Los cuatro sublenguajes de SQL

Sublenguaje	Para qué sirve	Sentencias
DDL · definición	Crear y modificar la estructura	`CREATE` `ALTER` `DROP` `TRUNCATE`
DML · manipulación	Consultar y modificar los datos	`SELECT` `INSERT` `UPDATE` `DELETE`
DCL · control	Dar y quitar permisos	`GRANT` `REVOKE`
TCL · transacciones	Confirmar o deshacer cambios	`COMMIT` `ROLLBACK` `SAVEPOINT`

`TRUNCATE` se clasifica como DDL porque vacía la tabla redefiniéndola, sin borrar fila a fila.

Un ejemplo de cada sublenguaje

```sql
CREATE TABLE socio (id_socio INTEGER PRIMARY KEY, nombre VARCHAR(50));   -- DDL
SELECT nombre FROM socio WHERE municipio = 'Murcia';                     -- DML
GRANT SELECT ON socio TO tecnico_junior;                                 -- DCL
COMMIT;                                                                  -- TCL
```

Reto: ¿en qué sublenguaje clasificarías `ALTER`, `REVOKE`, `UPDATE` y `SAVEPOINT`? Lo vas a necesitar en el juego 1.



Juego 1 · Clasifica y ordena

Puzle de **arrastrar y soltar** en tres niveles. Hay que superar uno para desbloquear el siguiente.

1. **Clasificación** de sentencias en DDL, DML, DCL o TCL.
2. **Ordena las cláusulas** de una consulta.
3. **Relaciona** cada expresión con su significado.

+10 por acierto **–2 por fallo** **10–15 min**

▶ Jugar

Al terminar, anota aquí tu llave:

1

Dígito 1

Segunda parte

Consultas con SELECT

Leer datos: **selección, proyección, orden y resumen**.



Estructura de `SELECT`

```sql
SELECT nombre, municipio     -- qué columnas quiero (proyección)
FROM socio;                  -- de qué tabla
```

nombre	municipio
Ana Martínez	Murcia
José López	Lorca
Carmen Sánchez	Cartagena
Pedro Ruiz	Murcia
Lucía Gómez	Cieza

Proyección, alias y `DISTINCT`

```sql
SELECT nombre,
       precio_kg AS euros_kg
FROM producto;
```

`AS` pone un **alias**: un nombre nuevo a la columna del resultado.

```sql
SELECT DISTINCT municipio
FROM socio;
```

municipio
Murcia
Lorca
Cartagena
Cieza

`DISTINCT` elimina los repetidos: Murcia sale **una** vez, aunque haya dos socios.

`WHERE`: filtrar filas

```sql
SELECT nombre, precio_kg
FROM producto
WHERE categoria = 'Hortaliza'
  AND precio_kg < 1.5;
```

nombre	precio_kg
Pimiento	1,20

Operadores

Comparación: `=` `<>` `<` `>` `⩽` `⩾`

Lógicos: `AND` `OR` `NOT`

Las comparaciones de texto van entre comillas simples: `'Hortaliza'`.

`BETWEEN`, `IN`, `LIKE` e `IS NULL`

```sql
-- BETWEEN: rango (con extremos)
WHERE kilos BETWEEN 200 AND 500

-- IN: una lista de valores
WHERE municipio IN ('Murcia', 'Lorca')
```

```sql
-- LIKE: patrón (% varios, _ uno)
WHERE nombre LIKE '%ez'

-- IS NULL: valor desconocido
WHERE fecha_alta IS NULL
```

`nombre LIKE '%ez'` devuelve los apellidos que acaban en *ez*: Martínez, López, Sánchez y Gómez, pero no Ruiz. Para comprobar si un valor falta usa `IS NULL`, nunca `= NULL`.

`ORDER BY`: ordenar el resultado

```sql
SELECT nombre, precio_kg
FROM producto
ORDER BY precio_kg DESC;
```

nombre	precio_kg
Albaricoque	2,10
Alcachofa	1,80
Melocotón	1,50
Pimiento	1,20
Limón	0,60
Naranja	0,45

`ASC` ordena de menor a mayor (es lo que se hace si no dices nada). `DESC`, de mayor a menor.

Se puede ordenar por **varias columnas**: `ORDER BY categoria, precio_kg DESC`.

Orden de escritura y orden lógico

Lo que escribes

1. `SELECT`
2. `FROM`
3. `WHERE`
4. `GROUP BY`
5. `HAVING`
6. `ORDER BY`

Lo que hace el SGBD

1. `FROM`
2. `WHERE`
3. `GROUP BY`
4. `HAVING`
5. `SELECT` (y `DISTINCT`)
6. `ORDER BY`

Por eso un **alias** definido en el `SELECT` se puede usar en el `ORDER BY` pero **no en el** `WHERE`: cuando se filtra, todavía no existe.

Consultas de resumen

Las **funciones de agregado** resumen muchas filas en un solo valor.

```
SELECT COUNT(*), SUM(kilos), AVG(kilos), MIN(kilos), MAX(kilos)
FROM entrega;
```

COUNT(*)	SUM(kilos)	AVG(kilos)	MIN(kilos)	MAX(kilos)
9	3050	338,9	100	800

Hay 9 entregas, que suman 3050 kg: una media de 338,9 kg por entrega.

`GROUP BY`: un resumen por grupo

```sql
SELECT id_socio, SUM(kilos) AS total
FROM entrega
GROUP BY id_socio;
```

id_socio	total
1	1050
2	950
3	700
4	350

Se forma **un grupo por cada valor** de `id_socio` y la función de agregado se calcula dentro de cada grupo.

`WHERE` frente a `HAVING`

`WHERE`

Filtra **filas**, *antes* de agrupar.

`WHERE kilos > 200` descarta las entregas pequeñas y después se agrupa: socio 1 pasa a sumar 850.

`HAVING`

Filtra **grupos**, *después* de agrupar.

`HAVING SUM(kilos) > 700` se queda con los socios que suman más de 700 kg.

```sql
SELECT id_socio, SUM(kilos) AS total
FROM entrega
GROUP BY id_socio
HAVING SUM(kilos) > 700;      -- resultado: socio 1 (1050) y socio 2 (950)
```

Los pasos de una consulta con grupos

1 · FROM
9 entregas

›

2 · WHERE
sin filtro: siguen las 9

›

3 · GROUP BY
4 grupos: socios 1, 2, 3 y 4

›

4 · HAVING
SUM > 700: quedan 2 grupos

›

5 · SELECT
id_socio y total

La consulta anterior, paso a paso. Si no te sale un resultado, **recorre las fases en este orden** y mira qué ocurre en cada una.

Tercera parte

Varias tablas

Las **composiciones** (*joins*) y las **subconsultas**.



Producto cartesiano

Si pides dos tablas sin condición, SQL combina **cada fila con cada fila**:

```sql
SELECT * FROM socio, entrega;      -- 5 socios × 9 entregas
```

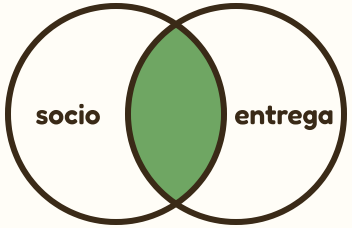
> **45 filas**, casi todas sin sentido: la entrega 1 de Ana aparece emparejada con Pedro, con Lucía… Falta decir **qué fila de una tabla corresponde con qué fila de la otra**.

Esa condición es la que añade el `JOIN`.

`INNER JOIN`: solo las que coinciden

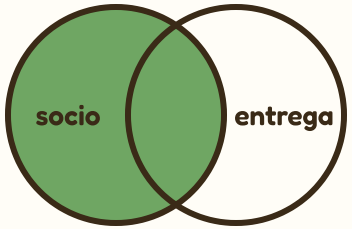
```sql
SELECT s.nombre, e.fecha, e.kilos
FROM socio s
INNER JOIN entrega e
  ON s.id_socio = e.id_socio
WHERE e.kilos ⩾ 500;
```

nombre	fecha	kilos
Ana Martínez	2026-03-02	500
José López	2026-03-05	800

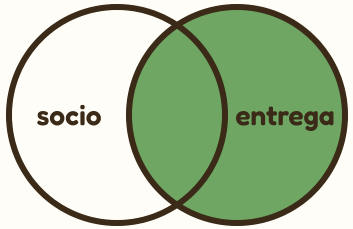


Solo las filas con pareja en **las dos** tablas. Si un socio no tiene entregas, no aparece.

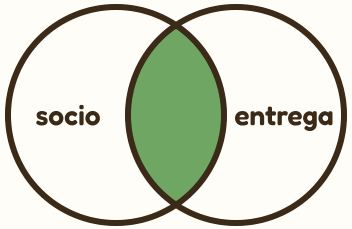
`LEFT` y `RIGHT JOIN`: las que no tienen pareja



LEFT JOIN: todas las de la tabla de la izquierda.



RIGHT JOIN: todas las de la tabla de la derecha.



INNER JOIN: solo las que coinciden.

Con un **outer join** las filas sin pareja salen igualmente, con `NULL` en las columnas de la otra tabla.

Ejemplo: la socia sin entregas

```sql
SELECT s.nombre, COUNT(e.id_entrega) AS entregas
FROM socio s
LEFT JOIN entrega e ON s.id_socio = e.id_socio
GROUP BY s.id_socio, s.nombre;
```

nombre	entregas
Ana Martínez	3
José López	2
Carmen Sánchez	2
Pedro Ruiz	2
Lucía Gómez	**0**

Lucía Gómez no ha hecho ninguna entrega, y con `INNER JOIN` habría desaparecido del resultado. Con `LEFT JOIN` sale con **0**.

Subconsultas escalares

Una subconsulta es una consulta **dentro de otra**. Si devuelve un solo valor, se puede usar en una comparación:

```
SELECT nombre
FROM producto
WHERE precio_kg > (SELECT AVG(precio_kg) FROM producto);
```

nombre
Alcachofa
Melocotón
Albaricoque

Primero se calcula el **precio medio** (1,275 €/kg) y después se buscan los productos que lo superan.

Subconsultas con `IN`, `NOT IN` y `EXISTS`

```sql
-- IN: alguna entrega > 400 kg
SELECT nombre FROM socio
WHERE id_socio IN
  (SELECT id_socio FROM entrega
   WHERE kilos > 400);
```

Resultado: **Ana Martínez** y **José López**.

```sql
-- NOT IN: sin ninguna entrega
SELECT nombre FROM producto
WHERE id_producto NOT IN
  (SELECT id_producto
   FROM entrega);
```

Resultado: **Albaricoque**.

`EXISTS` / `NOT EXISTS` comprueban si la subconsulta devuelve **alguna fila**. Con `NOT EXISTS` sobre las entregas de cada socio sale **Lucía Gómez**.

Subconsulta o `JOIN`: ¿cuál es mejor?

```sql
-- Con subconsulta
SELECT nombre FROM socio s
WHERE NOT EXISTS
  (SELECT 1 FROM entrega e
   WHERE e.id_socio = s.id_socio);
```

```sql
-- Con LEFT JOIN
SELECT s.nombre FROM socio s
LEFT JOIN entrega e
  ON s.id_socio = e.id_socio
WHERE e.id_entrega IS NULL;
```

Las dos devuelven **Lucía Gómez**. Un mismo problema se puede resolver de **varias formas**: valora la **legibilidad**, el **rendimiento** y los casos especiales (`NOT IN` falla si la subconsulta devuelve algún `NULL`).

Juego 2 · ¿Qué devuelve la consulta?

Concurso de preguntas contrarreloj. Cada pregunta muestra una consulta SQL: elige qué devuelve.

- **30 segundos** por pregunta.
- Botón **«Ver tablas»** para consultar los datos.
- Explicación y resultado real tras cada respuesta.

500–1000 puntos **+100 de racha** **15–20 min**

▶ Jugar

Anota aquí tu llave:

2

Dígito 2

Cuarta parte

Modificar datos

Insertar, actualizar y borrar, la integridad referencial y las **transacciones**.



`INSERT`: añadir filas

```sql
INSERT INTO socio (id_socio, nombre, municipio, fecha_alta)
VALUES (6, 'Luis Pérez', 'Totana', '2026-09-01');
```

También se puede insertar el **resultado de una consulta** con `INSERT … SELECT`:

```sql
CREATE TABLE resumen_categoria (
  categoria VARCHAR(30), precio_medio DECIMAL(5,2));

INSERT INTO resumen_categoria
SELECT categoria, ROUND(AVG(precio_kg), 2)
FROM producto GROUP BY categoria;
```

Resultado en la tabla nueva: Cítrico 0,53 · Fruta de hueso 1,80 · Hortaliza 1,50.

UPDATE y DELETE

```sql
UPDATE producto
SET precio_kg = precio_kg * 1.10
WHERE categoria = 'Cítrico';
```

Cambia el valor de las filas que cumplen la condición.

```sql
DELETE FROM entrega
WHERE id_entrega = 9;
```

Borra las filas que cumplen la condición.

> **Cuidado con el `WHERE`.** Si se te olvida, `UPDATE` cambia **todas** las filas y `DELETE` borra **todas** las filas de la tabla.



Integridad referencial

Una **clave ajena** obliga a que cada entrega pertenezca a un socio que existe.

```
DELETE FROM socio WHERE id_socio = 1;
-- ERROR: FOREIGN KEY constraint failed  (Ana tiene entregas)
```

Qué hacer al borrar o cambiar la fila «padre»

Se elige con `ON DELETE` y `ON UPDATE`:

`RESTRICT` / `NO ACTION`
Lo impide (por defecto).

`CASCADE`
Lo propaga a las filas hijas.

`SET NULL`
Pone `NULL` en la clave ajena.

Transacciones y propiedades ACID

Una **transacción** agrupa varias sentencias que se ejecutan **como una sola operación**.

A

Atomicidad

Se hace todo o no se hace nada.

C

Consistencia

Los datos siguen cumpliendo las reglas.

I

Aislamiento

Cada transacción va a lo suyo.

D

Durabilidad

Lo confirmado no se pierde.

COMMIT, ROLLBACK y SAVEPOINT

```sql
START TRANSACTION;

UPDATE entrega SET kilos = kilos - 50 WHERE id_entrega = 3;
SAVEPOINT tras_restar;
UPDATE entrega SET kilos = kilos + 50 WHERE id_entrega = 1;

ROLLBACK TO tras_restar;    -- deshace lo posterior al SAVEPOINT
COMMIT;                     -- confirma lo que queda
```

`COMMIT` confirma los cambios.

`ROLLBACK` los anula, en parte (`TO`) o del todo.

Bloqueos y concurrencia

¿Qué pasa si **dos personas** modifican la misma fila a la vez?

Bloqueos

- Un **bloqueo compartido** permite leer a varios a la vez.
- Un **bloqueo exclusivo** deja escribir solo a uno.
- El resto **espera** a que se libere.

Efectos

- Si dos transacciones se esperan entre sí, hay un **interbloqueo**.
- Los **niveles de aislamiento** dicen cuánto se ve de lo que hacen las demás.

Son solo nociones: lo importante es saber que el SGBD gestiona la concurrencia por ti.

Juego 3 · Terminal de la Cooperativa

Escape room con una consola SQL **real**: editor con resaltado de sintaxis y SQLite funcionando en tu navegador.

- **Seis incidencias** de la cooperativa.
- Escribe la sentencia y pulsa **Comprobar**.
- Al principio con **plantillas**; después, editor vacío.

500 por incidencia **pistas: −100** **30–40 min**

▶ Jugar

Anota aquí tus llaves:

3	4

Dígitos 3 y 4

Final

La caja fuerte

Reúne los **cuatro dígitos** y **salva la campaña**.



Abre la caja fuerte



1	2	3	4	5
6	7	8	9	0

Introduce los dígitos de las llaves, en orden: **juego 1, juego 2, juego 3 (dos dígitos)**.

¡Campaña salvada!

La copia de seguridad se ha recuperado y la **Cooperativa Huerta de Murcia** puede exportar su cosecha.

Has trabajado como un técnico o una técnica de bases de datos de verdad:

- Has **clasificado** sentencias SQL.
- Has **leído y predicho** consultas.
- Has **escrito SQL real** para resolver incidencias.

Insignia final

Maestro del DML

Y si has ganado las tres: **Clasificador** **Detective de consultas**

Lo que has aprendido

Consultar

- `SELECT`, `WHERE`, `ORDER BY`, `DISTINCT`
- `COUNT`, `SUM`, `AVG`, `GROUP BY`, `HAVING`
- `INNER`, `LEFT` y `RIGHT JOIN`
- Subconsultas con `IN`, `NOT IN` y `EXISTS`

Modificar

- `INSERT`, `UPDATE` y `DELETE`
- Integridad referencial
- `COMMIT`, `ROLLBACK` y `SAVEPOINT`
- Los cuatro sublenguajes: DDL, DML, DCL y TCL

> Un mismo problema tiene **varias soluciones**: elige la más clara y la más eficiente.

Autoevaluación

¿Qué has conseguido? Marca lo que **sabes hacer**:

- ☐ Clasificar sentencias en DDL, DML, DCL y TCL
- ☐ Filtrar y ordenar con `WHERE` y `ORDER BY`
- ☐ Resumir con agregados y `GROUP BY`
- ☐ Filtrar grupos con `HAVING`
- ☐ Combinar tablas con `JOIN`
- ☐ Resolver con subconsultas y compararlas
- ☐ Insertar, actualizar y borrar con cuidado
- ☐ Usar transacciones (`COMMIT` y `ROLLBACK`)

> Y también: ¿has perseverado ante los errores y has colaborado con tu equipo?



Sigue practicando en casa

Los juegos son páginas web abiertas: puedes **repetirlos** tantas veces como quieras.

Juego 1

Clasifica y ordena

Juego 2

¿Qué devuelve la consulta?

Juego 3

Terminal de la Cooperativa

Portal de la misión: antolin1.github.io/programacion-bbdd-uned

En el juego 3 puedes probar las versiones alternativas de cada consulta.

Créditos y bibliografía

Créditos

Recurso educativo gamificado para el módulo 0484 *Bases de datos*.

Presentación en Genially y tres juegos web propios (HTML, CSS y JavaScript). Tipografías **Fredoka** y **JetBrains Mono**; **sql.js** (SQLite en WebAssembly).

Bibliografía

- Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*.
- Chamberlin, D. D. y Boyce, R. F. (1974). *SEQUEL: A Structured English Query Language*.
- ISO/IEC 9075 (SQL).
- Silberschatz, Korth y Sudarshan. *Fundamentos de bases de datos*. McGraw-Hill.
- Elmasri y Navathe. *Fundamentos de sistemas de bases de datos*. Pearson.
- Werbach y Hunter (2012). *For the Win*. Wharton Digital Press.
- Real Decreto 405/2023, de 29 de mayo.